FaceMark

Attendance System

Product documentation — features, architecture, and operations.

Build date · 2026-06-15
Backend · Flask + SQLite + OpenCV LBPH
Audience · Operators, integrators, resellers

Table of contents

1. Product overview

FaceMark is a self-hosted, single-binary attendance platform built around face recognition. It runs on any PC with a webcam, requires no cloud account, and ships with every feature a school, college, or office needs out of the box — from in-browser live recognition to printable ID cards, a tablet kiosk mode, a hallway TV display, automated visitor logging, and the 75% defaulter rule that Indian colleges live by.

What problem does it solve?

Manual attendance is slow, error-prone, easy to proxy, and impossible to audit. Commodity biometric devices charge per-user license fees, lock data behind proprietary apps, and require expensive hardware. FaceMark replaces all of that with a single Flask app that runs on the school's own machine.

Who is it for?

- **Schools & colleges** — class-period attendance, defaulter reports, printable ID cards, guardian email exports
- **Corporate offices** — employee check-in / check-out, late-arrival flags, work-hour tracking
- **Coworking spaces, gyms, clinics** — member recognition, visitor security log
- **Government & PSU offices** — fully on-premise, no internet required after install

Key differentiators at a glance

Capability	What it does
Recognition backend	OpenCV LBPH (contrib) with face alignment, data augmentation, 3-of-5 vote confirmation
Crowd mode	Recognises every face in the frame simultaneously, per-identity voting, capture flash
Kiosk mode	Full-screen tablet UI with sound feedback for the school gate or office reception
Big-screen display	Projector / TV view with live counts and latest arrivals
Visitor log	Auto-snapshots every unknown face for security audit
ID cards	Printable cards with QR codes that link back to the student profile
Smart insights	Auto-detects late arrivals, attendance drops, birthdays, unknown faces
Defaulter rule	Configurable per-subject percentage threshold (default 75%)
Heatmap	GitHub-style 90-day grid per user
On-premise	SQLite, no cloud, no per-user fees, no internet needed

2. Architecture & technology stack

FaceMark is a monolithic Flask application that owns its own SQLite database and its own static asset directory. The browser is the only client; there is no separate React / mobile app to deploy.

Tech stack

Web framework	Flask 3 — sessions, jinja2 templates, MJPEG streaming
Database	SQLite (single file, facemark.db) with foreign keys enabled
Face detection	OpenCV Haar cascade with histogram equalisation
Face recognition	OpenCV LBPH (opencv-contrib-python) with KNN fallback
ML preprocessing	Eye alignment, blur rejection, data augmentation
Charts	Chart.js (CDN) for analytics page
QR codes	qrcode.js (CDN) for ID-card generation
Excel export	openpyxl via pandas ExcelWriter
Password hashing	werkzeug.security (PBKDF2-SHA256)
Styling	Bootstrap 5 base + custom CSS (~750 lines)

High-level data flow

A browser opens the Flask app. The MJPEG stream at `/video_feed` is served from the same Flask process that holds the OpenCV VideoCapture handle, so there is no separate inference service to orchestrate. The recogniser writes attendance directly into SQLite. Every dashboard panel polls a JSON API (`/api/stats`, `/api/recent`, `/api/just_captured`, `/api/insights`, `/api/heatmap`) so the UI updates within ~1 second of a recognition event without a page refresh.

Module map

app.py	Flask routes, auth, streaming pipeline, voting buffer, capture flash
db.py	SQLite schema, migrations, all CRUD + analytics queries
recognizer.py	LBPH/KNN abstraction with train/predict/is_trained API
face_utils.py	Haar detection, eye-based alignment, augmentation, blur gate
templates/	15 Jinja2 templates including the kiosk and display screens
static/css/styles.css	Custom design system (~750 lines)
static/faces/	Captured face crops per user (training data)
static/profiles/	160x160 profile thumbnails extracted at enrollment

static/visitors/	Auto-snapshots of unknown faces
Attendance/	CSV files (export-only, kept for backward compatibility)

3. Authentication & roles

Every route except `/login` is protected by a `@login_required` decorator. Sessions use Flask's signed-cookie store with a configurable secret (`FACEMARK_SECRET` environment variable).

Default credentials

On first run the database seeds a single admin: `admin / admin123`. The Settings page forces a password change in production. Passwords are stored as PBKDF2-SHA256 hashes via werkzeug.security, never plain text.

Audit log

Every admin action is recorded in the `audit_log` table: login, logout, add/edit/delete user, add/delete subject, create/delete session, set active session, update settings, clear visitor log, manual mark, bulk import. Visible at `/audit`.

4. Recognition pipeline (LBPH + crowd mode)

The pipeline is engineered for accuracy in real-world conditions — mixed lighting, multiple faces, partial occlusion, motion.

Step 1 — detection

- Haar cascade with `scaleFactor=1.15`, `minNeighbors=6`, `minSize=50×50` (small enough for the back of a classroom)
- Histogram equalisation runs before detection — robust to backlit windows and dim corridors

Step 2 — preprocessing

- Eye-based alignment: a second Haar cascade locates eyes, the crop is rotated so the eye line is horizontal
- Resized to 200×200, converted to grayscale, histogram-equalised
- Blur gate at enrolment: Laplacian variance must exceed 60 or the frame is rejected. The kiosk shows the live *Sharpness* number so users know when to hold still

Step 3 — enrolment & training

- 25 sharp samples per user
- Each sample is augmented to 5 variants (flip + brightness ±20 + rotation ±5°) before training
- LBPH parameters: `radius=1, neighbors=8, grid_x=8, grid_y=8`
- Auto-fallback to scikit-learn KNN if `cv2.face` is unavailable

Step 4 — crowd-mode voting

For every frame, the pipeline predicts every detected face independently. Each accepted prediction is appended to a per-identity ring buffer with a timestamp. A 2.5-second sliding window prunes stale votes. An identity is confirmed only when it accumulates **3 hits inside that window**, after which an 8-second cooldown prevents re-marking. This makes the system robust to false positives even with

10+ people on screen.

Step 5 — capture flash

For 1.5 seconds after marking, the confirmed face gets a green ✓ OK circle, a CAPTURED label, and a darker green bounding box. The browser also fires a non-blocking toast — critical visual feedback so users at the kiosk know they're done.

Step 6 — unknown face logging

Any face whose nearest LBPH distance exceeds the configurable threshold (default 80) is labelled *Unknown* and a 200×200 snapshot is written to `static/visitors/`. A 30-second cooldown prevents storage flooding.

5. Core attendance flow

Attendance is automatic, idempotent, and bidirectional — first sighting of the day is recorded as a check-in, last sighting (after the configured minimum gap) is recorded as a check-out.

Check-in

- First confirmed sighting of the day for a person inserts a row into `attendance`
- Compared against `work_start_time + late_threshold_min`; past that, `is_late = 1`
- Tagged with the current session id if a class session is active

Check-out

- A subsequent sighting of the same person, at least `min_checkout_gap_min` minutes after their check-in, updates the `check_out` column
- Allows the daily report to show real work hours / class hours

Late-arrival flagging

Configurable in Settings. The dashboard “Late today” KPI, the history page status chip, the heatmap colour (amber), the smart-insights strip, and the big-screen display all use this single flag.

Manual mark / override

Camera missing? Sick student excused? Every user list shows a green check button for instant manual check-in, and every user-detail page has explicit *Check in* and *Check out* buttons. Manual marks are recorded in the audit log with the admin’s username.

Yet-to-arrive panel

A live panel on the dashboard shows every registered user with no check-in row for today. One-click manual mark on each row — useful when a few students arrive while the camera is busy, or for late reconciliation at end of day.

Live feed

A separate panel polls `/api/recent` every 3 seconds and shows the eight most recent sightings with profile thumbnails, time, and late/on-time chip. Updates without page refresh.

6. Academic model — Subjects, Sessions, Defaulters

A flat daily attendance is enough for an office, but colleges need period-level granularity. FaceMark supports that without disturbing the simple daily flow.

Subjects

Stored at `/subjects`. Each subject has a name, an optional code (e.g. `CS101`), and an optional department/class link.

Class sessions

A session is a single class period: *subject* + *date* + *start_time* + *end_time* + *notes*. Created at `/sessions`. Sessions can be created on the fly with the *Activate immediately* checkbox — typical workflow for a teacher walking into class.

Active session

At most one session can be “active” at a time. While active, every confirmed recognition writes *two* rows: the daily check-in (`attendance`) *and* the period attendance (`session_attendance`). The dashboard shows a pulsing green banner with *End session* button so it’s obvious when class is in session.

Defaulter report

At `/defaulters`, FaceMark computes per-student per-subject attendance percentages and lists everyone below the configurable threshold (default 75%). Colour-coded pills (red below 50%, amber below threshold) make it easy to act on. The threshold is configurable on the page itself or in Settings.

Printable register

At `/register/print`, FaceMark renders a print-friendly attendance register for any date — daily check-ins, sessions held, signature lines for the faculty and HOD, page numbers, and auto-trigger of the browser print dialog. Exactly what colleges hand to inspectors and university auditors.

7. Unique selling features

These are the features that turn FaceMark from “a face-recognition attendance system” into **a product you can sell at a premium**.

Kiosk mode — the gate terminal

Available at `/kiosk`. Full-screen, no-chrome, sound-on. Auto-starts the recognition pipeline, plays a two-tone chime when someone is marked, shows their profile photo + name + class for ~4 seconds in a giant card, and keeps a live clock and present/late counters in the corners. Designed to run unattended on a ■15,000 tablet mounted at the school gate or office reception.

Big-screen display — the hallway TV

Available at `/display`. Designed for a TV in the hallway or a projector at assembly. Shows gigantic counters (Present today, Attendance rate), the live clock, and a “Latest arrivals” list with profile photos. Refreshes every 3 seconds with no user input. Strong marketing visual for school open days and parent visits.

Visitor log — entrance security

Available at `/visitors`. Every face detected that does not match anyone in the database is silently saved as a 200×200 snapshot with timestamp and camera name. Schools and offices need this for safety compliance (CCTV-like log) without paying for separate surveillance software. One-click *Clear log* deletes all snapshots from disk to keep storage in check.

Smart insights — zero-effort alerting

A strip of automatic alerts at the top of the dashboard. The engine detects, every load:

- **Late arrivals today** — count + link to today’s view
- **Yet to arrive** — registered users with no check-in today
- **Unknown faces in last 24 h** — link to visitor log
- **Attendance drop** — per-user: this week vs last week, flagged if recent < previous – 1
- **Birthdays today** — deduced from `date_of_birth`

Printable ID cards — a revenue line

Available at `/idcard/<roll>`. A 340×540 pixel card with org-branded banner, profile photo, roll number, department, email, date of birth, enrolment date, and a QR code that links back to the student’s profile page. One-click print. Schools sell ID cards every year — FaceMark turns that into a built-in feature instead of a printer-vendor headache.

GitHub-style 90-day heatmap

On every user profile, a 30-column grid shows the last 90 days of attendance. Green = on time, amber = late, grey = absent. Instant visual demo wow-factor; parents and inspectors understand it without explanation.

Birthday widget

Today’s birthdays appear in the smart-insights strip with a cake icon and a link to the user’s profile. Drives daily admin login and parent engagement.

8. All routes (full URL map)

Every URL the app responds to, grouped by category.

Auth & shell

`GET /login`	Login screen
`POST /login`	Validate credentials, set session
`GET /logout`	Clear session, return to login
`GET /`	Dashboard

Recognition stream

`GET /video_feed`	MJPEG live stream
`POST /start_recognise`	Switch streaming pipeline to recognise mode
`POST /stop_capture`	Reset pipeline to idle
`GET /capture_status`	JSON: current capture progress

Users

`POST /add`	Begin enrolment + start capture
`GET /register_capture`	Capture page with sharpness bar
`GET /listusers`	Filterable user list
`POST /edituser`	Update name / dept / email / DOB
`GET /deleteuser`	Delete user, retrain model
`POST /import_users`	Bulk CSV import
`GET /user/<pid>`	Profile + heatmap + history

Manual override

`POST /manual_mark`	Force check-in or check-out

History & exports

`GET /history`	Pick any date and view its attendance
`GET /download`	CSV or .xlsx download (?fmt=xlsx)
`GET /register/print`	Print-ready attendance register
`GET /contacts.csv`	Bulk contact export (incl. guardian email)

Academic model

`GET/POST /subjects`	List + add + delete subjects

`GET/POST /sessions`	List + add + delete class sessions
`GET /session/<sid>`	Per-session attendance roster
`POST /set_active_session`	Activate or clear active session
`GET /defaulters`	Students below threshold percentage

Operate (kiosk & display)

`GET /kiosk`	Full-screen tablet kiosk
`GET /display`	Big-screen hallway display
`GET /visitors`	Unknown-face log
`POST /visitors/clear`	Wipe visitor log + snapshots
`GET /idcard/<pid>`	Printable ID card with QR

Reports

`GET /analytics`	Chart.js dashboards: trend + late vs on-time + top attenders

Configuration

`GET/POST /settings`	Org, working hours, recognition threshold, departments, admin password
`GET /audit`	Audit log viewer

JSON APIs

`GET /api/stats`	Counts for dashboards
`GET /api/recent`	Last 8 sightings today
`GET /api/just_captured`	Live capture-flash events
`GET /api/insights`	Smart-insights payload
`GET /api/heatmap/<pid>`	90-day attendance grid

9. Database schema

SQLite is the single source of truth. Every CSV / Excel export is generated on demand from these tables. Lightweight migrations run in `db.init_db()` so future schema bumps are non-destructive.

Table	Columns
admin_users	id, username (unique), password_hash, role, created_at
departments	id, name (unique). Used as classes for schools.
persons	id, person_id (roll/employee), name, department_id, email, guardian_email, date_of_birth, created_a
attendance	id, person_id, date, check_in, check_out, is_late, UNIQUE(person_id, date)
settings	key (PK), value. Stores org_name, work_start_time, late_threshold_min, recognition_threshold, atte
audit_log	id, actor, action, detail, created_at (indexed DESC)
subjects	id, name, code, department_id, UNIQUE(name, department_id)
class_sessions	id, subject_id, date, start_time, end_time, notes, created_at (date indexed)
session_attendance	id, session_id, person_id, marked_at, UNIQUE(session_id, person_id)
visitors	id, snapshot, seen_at (indexed DESC), camera

Migrations

Two safe `ALTER TABLE`s run idempotently on startup:

- Add `persons.guardian_email` if missing
- Add `persons.date_of_birth` if missing

10. Files & folders

Path	Purpose
app.py	Flask routes + recognition streaming + voting + auth
db.py	SQLite schema + 30+ helper functions
recognizer.py	LBPH/KNN backend abstraction
face_utils.py	Detection, alignment, augmentation, blur gate
build_docs.py	This documentation generator
requirements.txt	Flask, opencv-contrib-python, scikit-learn, joblib, pandas, openpyxl, werkzeug
facemark.db	SQLite database (auto-created at first run)
haarcascade_frontalface_default.xml	OpenCV face cascade
templates/	15 Jinja2 templates
static/css/styles.css	Custom design system
static/faces/<Name_RollNo>/	Face crops captured at enrolment
static/profiles/<RollNo>.jpg	160x160 profile thumbnails
static/visitors/visitor_<ts>.jpg	Unknown-face snapshots
static/lbph_model.yml	Trained LBPH model
static/lbph_labels.json	Label ↔ user mapping
Attendance/Attendance-<tag>.csv	Legacy CSVs (export-only path)
docs/FaceMark-Documentation.pdf	This document

Template inventory

Template	Purpose
base.html	Sidebar shell + topbar + toast deck
login.html	Standalone login screen
home.html	Dashboard: insights, KPIs, live recognition, register form, today’s attendance, live feed, yet-to-ar
listusers.html	Filterable user list with edit modal + bulk import
register.html	Live capture page with sharpness bar
history.html	Date-picker history + CSV / Excel / print
analytics.html	Chart.js trend + late vs on-time + top attenders

Template	Purpose
settings.html	Org, working hours, recognition, departments, password
audit.html	Audit log viewer
subjects.html	Subject CRUD
sessions.html	Class-session CRUD with Activate-now option
session_detail.html	Per-session attendance roster
defaulters.html	Below-threshold student list
user_detail.html	Profile + KPIs + heatmap + history + ID card link
visitors.html	Unknown-face snapshot grid
register_print.html	Print-ready attendance register
kiosk.html	Full-screen tablet kiosk with audio
display.html	Big-screen hallway display
idcard.html	Printable ID card with QR code

11. Settings reference

All settings live in the `settings` table and can be edited at `/settings`.

Key	Description & default
org_name	Displayed in sidebar, login screen, footer, ID cards. Default: FaceMark Attendance.
work_start_time	HH:MM. Default 09:00. Anything past this plus grace minutes is flagged late.
late_threshold_min	Minutes of grace after work_start. Default 15.
min_checkout_gap_min	Minimum minutes between a check-in and a check-out for the same person. Default 30
recognition_threshold	LBPH distance cutoff. Lower = stricter. Default 80 (LBPH); use 7000 with KNN.
attendance_required_pct	Percentage threshold for defaulter report. Default 75 (Indian college rule).
active_session_id	Currently-running class session id. Empty string means daily-attendance mode.
logo_filename	Optional uploaded organisation logo.

Tuning recognition for your environment

- **Bright outdoor light + multiple similar-looking users:** drop threshold to 60–70 to avoid mistakes
- **Dim indoor light, very few users:** raise threshold to 90–95 so the system still recognises them
- **Want fewer late flags?** Raise *late_threshold_min* to 30
- **Want shorter check-out gap?** Drop *min_checkout_gap_min* to 5 — useful in a gym where members enter and leave quickly

12. Day-to-day workflows

A. First-time setup (10 minutes)

- Install: `pip install -r requirements.txt`
- Launch: `python app.py`
- Browse to `http://localhost:5000/`; sign in with `admin / admin123`
- Settings → change admin password, set org name and work hours
- Settings → add classes / departments (e.g. *B.Tech CSE Sem 4*)
- Subjects → add subjects with codes (e.g. *CS101 Data Structures*)
- Users → bulk-import students via CSV (name, id, department, email, guardian_email)
- Dashboard → register the first face to train the model

B. Daily teacher workflow

- Open dashboard → Sessions → New session with *Activate immediately* checked
- Click Start on live recognition; students walk in, get marked
- Click *End session* when class is over
- Per-session attendance is now visible at `/session/<id>`

C. Daily reception / gate workflow

- Power on tablet at the gate; browser opens to `/kiosk` in fullscreen
- Recognition starts automatically; chime on every mark
- No human attendant required

D. End-of-day admin workflow

- History → today's date → Excel / Print
- Defaulters → export below-threshold list
- Visitor log → review unknown faces from the day; clear if all benign

E. End-of-term reporting workflow

- Analytics → Last 30 days view
- Defaulters → set threshold to 75% → download list
- Per-student: `/user/<roll>` → prints easily for parent meetings
- ID card renewals: `/idcard/<roll>` for each student → print to card stock

13. Sales pitch & target buyers

A short one-liner that works in cold emails, demos, and proposals:

"*FaceMark is the only attendance system that ships with everything a school needs out of the box — a tablet kiosk for the gate, a hallway TV display, automatic visitor logging, printable ID cards, smart insights that flag attendance drops before parents complain, and the 75% defaulter rule with one-click guardian email export. Setup is under ten minutes, no cloud account needed, works on a single PC.*"

Comparison vs. typical competitors

Capability	Typical competitor	FaceMark
Per-user license fees	Yes	No — unlimited users
Cloud required	Often	No — fully on-premise
Multi-face / crowd recognition	Few	Yes
Kiosk + TV display included	Rare	Yes
Visitor log	Add-on	Built-in
ID card generator	Vendor lock-in	Built-in PDF/print
Per-class period attendance	Rare	Yes
75% defaulter rule	Manual	One-click
Heatmap visualisation	No	Yes
Audit log for compliance	Sometimes	Yes, every action

Recommended target buyers

- **Private schools (CBSE/ICSE)** — willing to pay for parent engagement features; ID card revenue line
- **Engineering colleges** — 75% defaulter rule, per-subject attendance, university-grade printable register
- **Coaching institutes** — batch-wise sessions, late tracking, guardian email integration
- **Corporate offices** — employee check-in/out, work-hour audit, visitor security log
- **Government & PSU offices** — on-premise requirement; no cloud dependency
- **Gyms & coworking spaces** — member recognition; visitor log for safety

14. Setup & install guide

Hardware

- Any Windows / macOS / Linux PC with 4 GB RAM and a webcam
- Optional: Android tablet for kiosk mode (any browser)
- Optional: TV for the big-screen display (HDMI from the PC)

Software prerequisites

- Python 3.10 or newer
- pip + a working internet connection for the initial install only

Install steps

```
pip install -r requirements.txt
python app.py
open http://localhost:5000/ ; login admin / admin123
```

Production deployment notes

- Replace the Flask dev server with gunicorn or waitress
- Place behind nginx with HTTPS terminated at the edge
- Set `FACEMARK_SECRET` env var to a strong random value before first boot
- Schedule nightly backups of `facemark.db` and `static/faces/`
- Restrict the kiosk URL to a single IP if the tablet is on the same LAN

15. Future roadmap

Features the codebase is already structured to support, in priority order:

Feature	Notes
Eye-blink anti-spoofing	Detect liveness before marking; defeats printed-photo attacks. Hooks into <font face="Courie
WhatsApp / SMS guardian alerts	Daily attendance and absentee push notifications. guardian_email</fon
Mobile companion PWA	Parents see their child’s heatmap and history. The JSON APIs are already in place.
Multi-tenant SaaS mode	Subdomain-per-school, central billing. Database is per-org-keyed today.
Recurring weekly timetable	Schedule Monday–Friday class slots once; sessions auto-create.
Excused leave / leave requests	A new leaves table + approval flow.
Dockerfile + gunicorn + nginx	One-command production deployment.
DNN face detector option	Better recall in dense crowds (30 px faces). 5 MB model download.

End of document.